ALGORITMOS DE ORDENAMIENTO DE LISTAS CON ESTRATEGY

Patron de Diseño: *Strategy*

Selección dinámica de algoritmos de ordenamiento según el tamaño de la colección y delegación polimórfica

Autor: **Angie Nathaly Cisneros Guerrero**

20242020084

Fecha: Septiembre 2026

Índice

1. Diagrama UML **2**

2. Explicación de las Clases **3**
2.1. Interfaz Estrategia (AlgoritmoOrdenamiento.java) 3
2.2. Clase Contexto (Contexto.java) 3
2.3. Fábrica de Estrategias (FabricaSimpleEstrategia.java) 5
2.4. Estrategias Concretas (Algoritmos de Ordenamiento) 5

3. Código de Cliente **7**
3.1. Clase Cliente (Cliente.java) 7

4. Pruebas y Ejecución **9**
4.1. Bubble Sort 10
4.2. Selection Sort 11
4.3. Insertion Sort 11
4.4. Quick Sort 12
4.5. Heap Sort 13
4.6. Merge Sort 14

5. Repositorio **15**
5.1. Enlace GitHub 15

Diagrama UML

Descripción General

El proyecto implementa una arquitectura combinada utilizando el patrón de comportamiento **Strategy** junto con una fábrica creacional **Simple Factory**. El sistema permite desacoplar la lógica de los algoritmos de ordenamiento (`BubbleSort`, `SelectionSort`, `InsertionSort`, `QuickSort`, `HeapSort` y `MergeSort`) de la clase cliente, permitiendo además la asignación y conmutación dinámica de estrategias a través de un objeto `Contexto`.

Diagrama de Clases

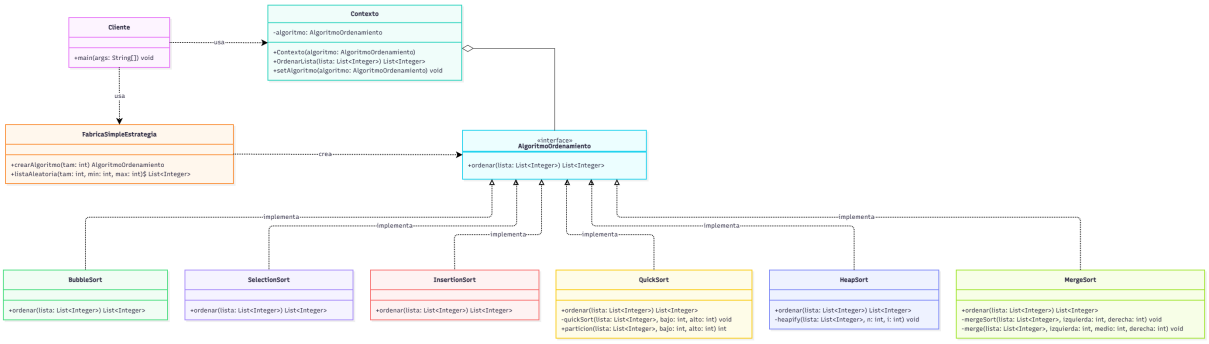


Figura 1: Diagrama de Clases del Sistema (Patrón Strategy y Fábrica).

Explicación de las Clases

Interfaz Estrategia (AlgoritmoOrdenamiento.java)

Descripción

Define el contrato común que deben cumplir todas las estrategias concretas de ordenamiento. Expone el método `ordenar` que recibe una lista de números enteros y retorna otra lista organizada.

Código Fuente: AlgoritmoOrdenamiento.java

```java
package aordenamiento.estrategy;

import java.util.List;

public interface AlgoritmoOrdenamiento {

    List<Integer> ordenar(List<Integer> lista);

}
```

Clase Contexto (Contexto.java)

Descripción

Mantiene una referencia a un objeto `AlgoritmoOrdenamiento`. Es el encargado de delegar la operación de ordenamiento hacia la estrategia activa mediante polimorfismo. Además, expone un método modificador (`setAlgoritmo`) que permite cambiar o sustituir la estrategia asociada durante el tiempo de ejecución.

Código Fuente: Contexto.java

```java
package aordenamiento.estrategy;

import java.util.List;

public class Contexto {

    private AlgoritmoOrdenamiento algoritmo;


    public Contexto(AlgoritmoOrdenamiento algoritmo) {
        this.algoritmo = algoritmo;
    }

    public List<Integer> OrdenarLista(List<Integer> lista){
        return algoritmo.ordenar(lista);
    }

    public void setAlgoritmo(AlgoritmoOrdenamiento algoritmo) {
        this.algoritmo = algoritmo;
    }
```

```
21 }
```

Fábrica de Estrategias (FabricaSimpleEstrategia.java)

Descripción

Encapsula la lógica de creación de los algoritmos de ordenamiento (`crearAlgoritmo`) evaluando el tamaño de los datos (`tam`), retornando la estrategia óptima, también incluye un método para generar colecciones numéricas aleatorias.

Código Fuente: FabricaSimpleEstrategia.java

```java
package aordenamiento.estrategy;

import java.util.ArrayList;
import java.util.List;
import java.util.Random;

public class FabricaSimpleEstrategia {

    public AlgoritmoOrdenamiento crearAlgoritmo(int tam){
        if (tam < 10) {
            return new BubbleSort();
        } else if (tam < 50) {
            return new SelectionSort();
        } else if (tam < 1_000) {
            return new InsertionSort();
        } else if (tam < 10_000) {
            return new QuickSort();
        } else if (tam < 50_000) {
            return new HeapSort();
        } else {
            return new MergeSort();
        }
    }

    public static List<Integer> listaAleatoria(int tam, int min,
       int max) {
        Random random = new Random();
        List<Integer> lista = new ArrayList<>(tam);

        for (int i = 0; i < tam; i++) {
            lista.add(random.nextInt(max - min) + min);
        }

        return lista;
    }
}
```

Estrategias Concretas (Algoritmos de Ordenamiento)

Descripción

Las estrategias concretas corresponden a las clases que implementan la interfaz `AlgoritmoOrdenamie`
Cada una encapsula un algoritmo específico de ordenamiento estructurado para operar sobre copias de las listas ingresadas, garantizando la inmutabilidad de los datos originales. Los algoritmos implementados son:

- **BubbleSort**: Ideal para listas sumamente pequeñas; realiza comparaciones adyacentes sucesivas intercambiando elementos desordenados.
- **SelectionSort**: Selecciona el elemento menor de la sublista no ordenada y lo ubica al inicio en cada iteración, minimizando la cantidad de intercambios.
- **InsertionSort**: Inserta de manera ordenada cada elemento en su posición correcta dentro de una sublista previa, destacando en colecciones pequeñas o semi-ordenadas.
- **QuickSort**: Algoritmo eficiente de tipo divide y vencerás basado en el uso de un pivote y particionamiento recursivo para listas de mediano y gran volumen.
- **HeapSort**: Construye un árbol binario parcial (Max-Heap) para extraer iterativamente el elemento mayor de manera eficiente y con consumo controlado de memoria.
- **MergeSort**: Utiliza el enfoque divide y vencerás dividiendo recursivamente la lista a la mitad y fusionando de forma ordenada sublistas, garantizando estabilidad y un rendimiento óptimo en volúmenes masivos.

Código de Cliente

Clase Cliente (Cliente.java)

Descripción

La clase `Cliente` solicita al usuario el tamaño de la lista a través de la consola mediante `Scanner` validando posibles errores de entrada. Utiliza la fábrica para obtener los datos aleatorios y la estrategia inicial, e interactúa con el `Contexto` para ejecutar el ordenamiento.

Código Fuente: Cliente.java

```
/**
 *
 * @author natha
 */
public class Cliente {


    /**
     * @param args the command line arguments
     */
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.println("--- Ordenamiento de listas ---");

        int tam=0;
        while(tam==0){
           System.out.println("Ingrese el tamano: ");
            try{
            tam=sc.nextInt();

            }catch(InputMismatchException e){
                System.out.println("Error: el tamano debe ser un
                   numero entero");
                sc.next();
            }


        }
        sc.close();

        FabricaSimpleEstrategia  objFabrica = new
           FabricaSimpleEstrategia();
        List<Integer> lista = objFabrica.listaAleatoria(tam
           ,0,1000);

        AlgoritmoOrdenamiento objAlgoritmo = objFabrica.
           crearAlgoritmo(tam);
        Contexto objContexto = new Contexto(objAlgoritmo);
        List<Integer> listaOrdenada =objContexto.OrdenarLista(
           lista);
```

```
36
37          System.out.println("\nAlgoritmo␣de␣Ordenamiento␣usado:␣"+
                objAlgoritmo.getClass().getSimpleName());
38
39          System.out.println("\nLista␣Aleatoria␣Generada:␣\n"+lista
                );
40
41          System.out.println("\n\nLista␣Ordenada:\n"+listaOrdenada)
                ;
42      }
43
44  }
```

Flujo de Ejecución

1. El usuario ingresa un valor numérico entero correspondiente al tamaño de la colección.
2. La fábrica genera una lista de números enteros aleatorios no ordenados.
3. Se evalúa el tamaño de la lista para instanciar la estrategia de ordenamiento óptima mediante la fábrica.
4. El cliente inyecta la estrategia en el `Contexto` (y permite la modificación mediante su método `setAlgoritmo`).
5. El contexto ejecuta el método `ordenar` sobre la lista, devolviendo el resultado final.

Pruebas y Ejecución

Nota sobre las pruebas: Adicionalmente a la ejecución principal del cliente, el repositorio cuenta con una carpeta que incluye clientes de prueba específicos e independientes (como clientes de validación para BubbleSort, SelectionSort y pruebas unitarias de rendimiento) diseñados para verificar de manera aislada el correcto funcionamiento de cada uno de los algoritmos implementados.

Tamaño diferente de número

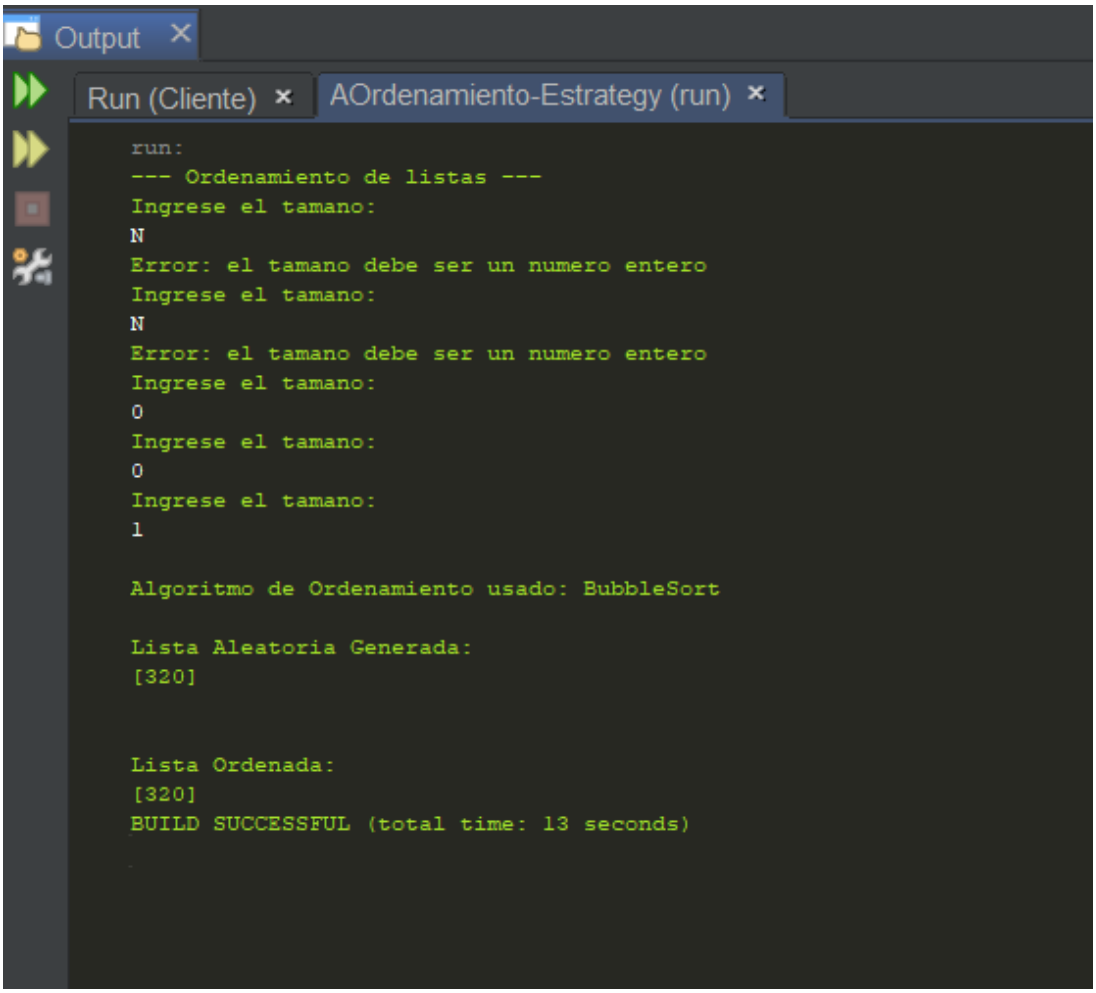


Figura 2: Salida Consola.

Bubble Sort

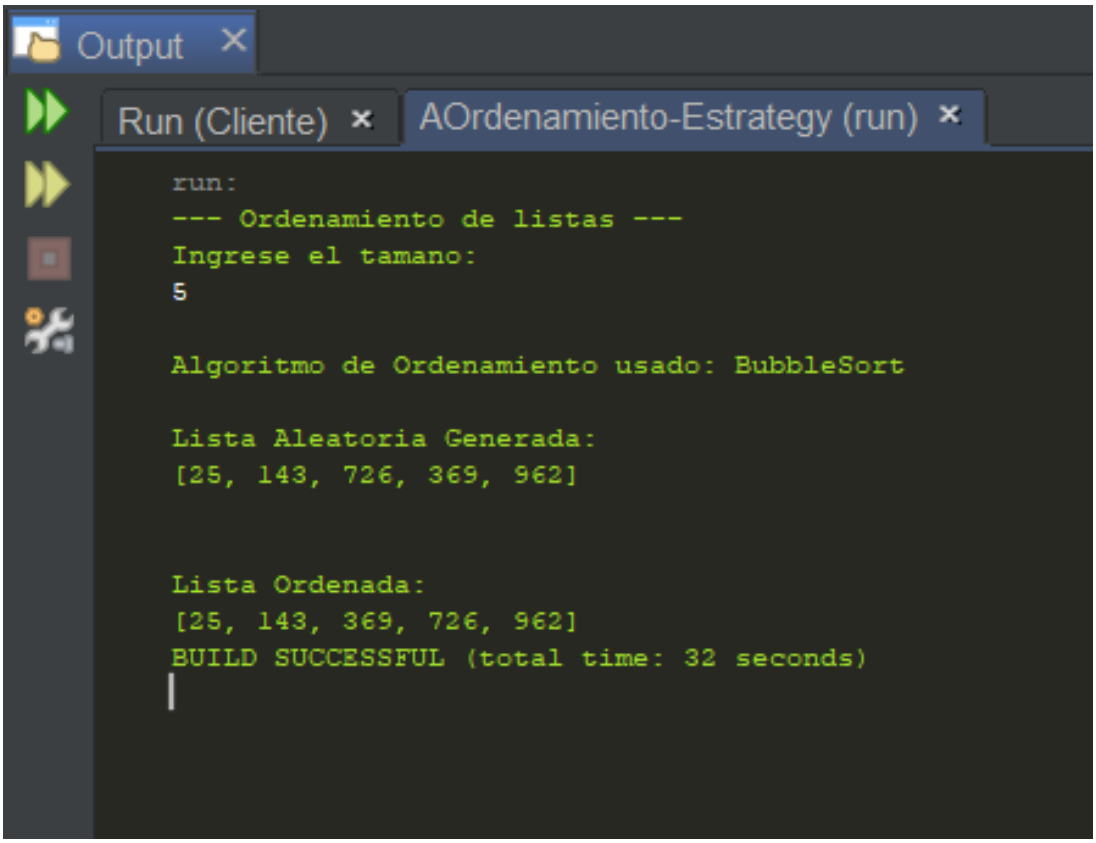


Figura 3: Salida Consola.

Selection Sort

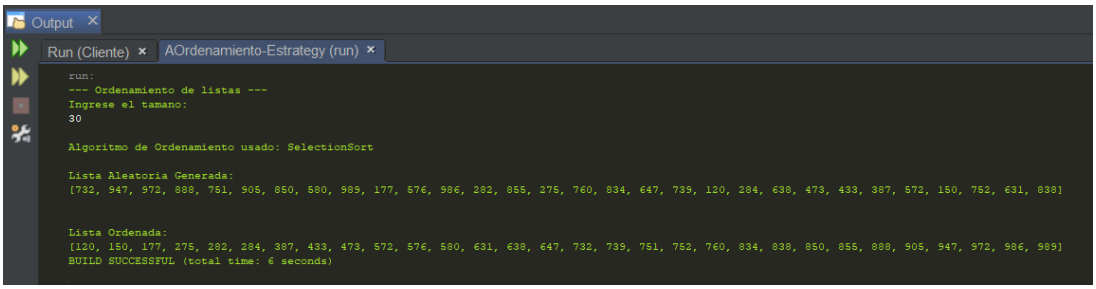


Figura 4: Salida Consola.

Insertion Sort

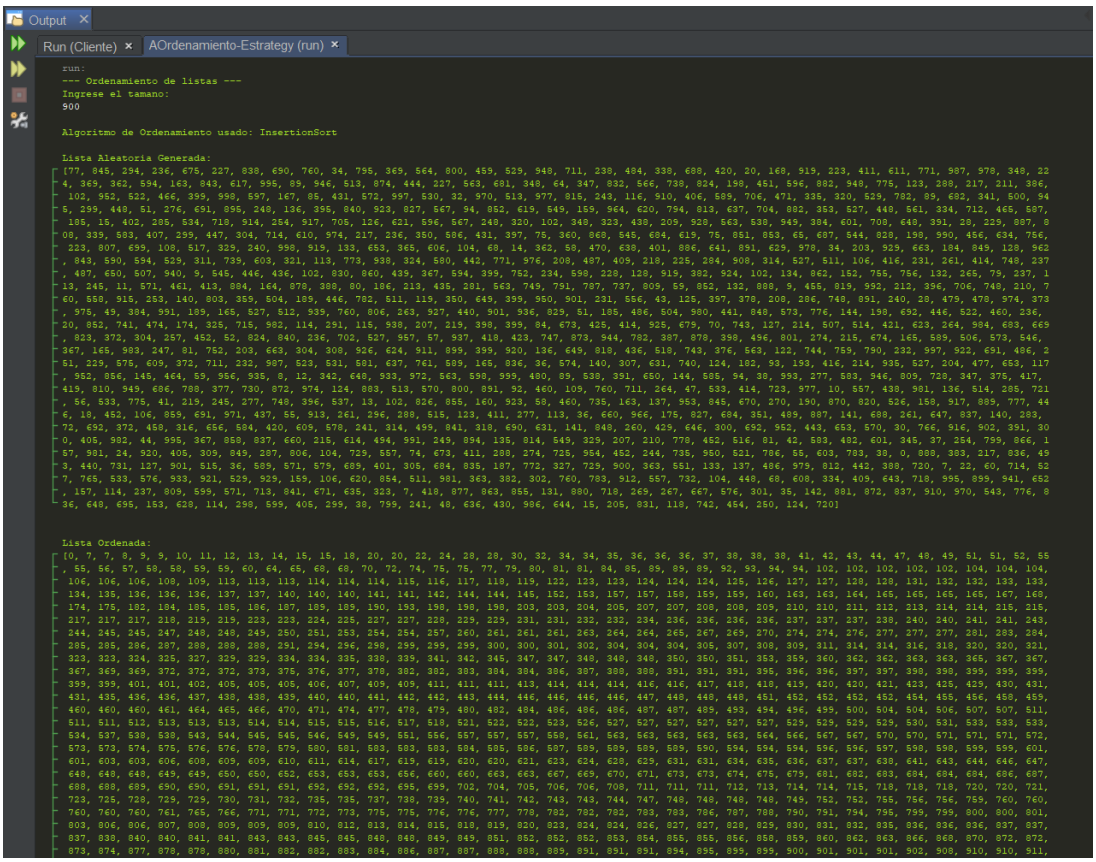


Figura 5: Salida Consola.

Quick Sort

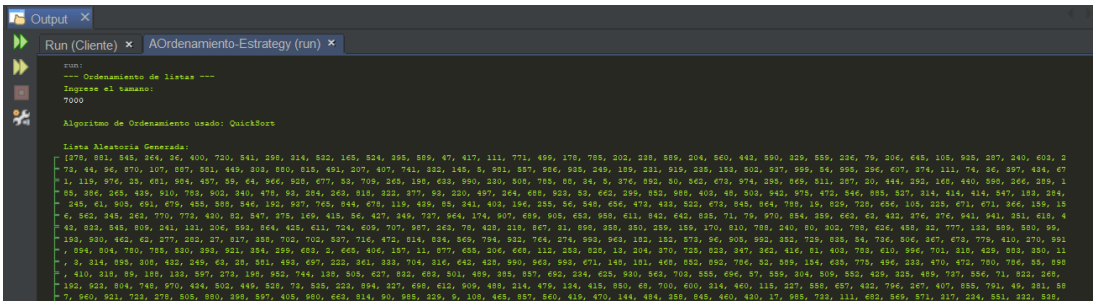


Figura 6: Salida Consola.

Heap Sort

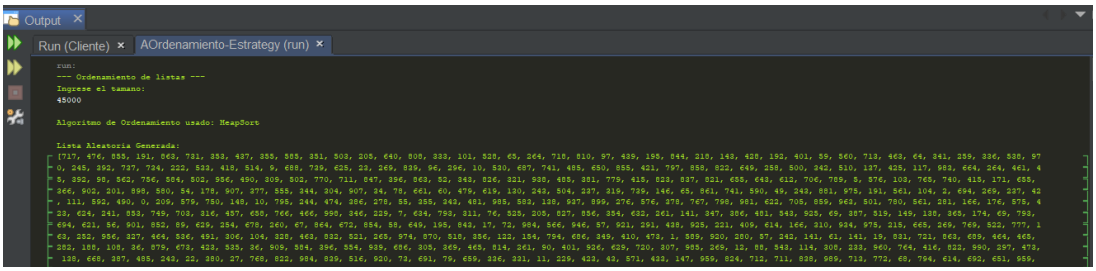


Figura 7: Salida Consola.

Merge Sort

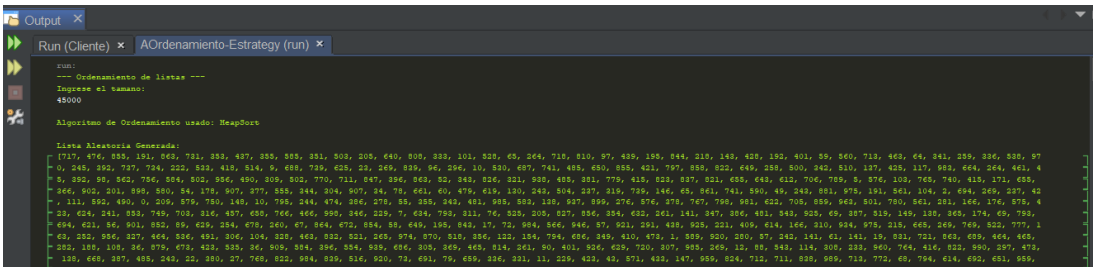


Figura 8: Salida Consola.

Repositorio

Enlace GitHub

- https://github.com/Thalyhx/Modelos-de-Programacion/tree/main/Patrones%20Comportamiento/AOrdenamiento-Strategy